Food Delivery Platform - Complete Final Documentation

1. Project Title

Food Delivery Platform
A full-stack food delivery system inspired by on-demand delivery platforms such as Mr D, built with a Spring Boot backend and separate frontend applications for customers, restaurant owners, and drivers.

---

2. Executive Summary

This project implements a role-based food delivery platform that supports the full lifecycle of a food order, from browsing restaurants and placing an order to restaurant processing, driver assignment, delivery updates, and final completion.

The solution is divided into four major parts:

1. **Backend API** for authentication, business logic, persistence, and role-based access control
2. **Customer Web App** for browsing restaurants, ordering, and tracking orders
3. **Restaurant Console** for restaurant-specific order and menu management
4. **Driver App** for accepting deliveries and updating delivery progress

The system was tested successfully end to end. A customer can place an order, the correct restaurant can view and manage that order, and a driver can accept and complete the delivery while the status changes are reflected across the system.

---

3. Problem Statement

Modern food delivery systems require the coordination of multiple actors:

- customers who browse and place orders
- restaurants that prepare orders
- drivers who deliver those orders
- a backend system that orchestrates all operations securely and reliably

Without a structured digital platform, these interactions become inefficient, difficult to track, and prone to communication failures. This project addresses that problem by

providing a centralized system with clear role separation, secure login, order management, and delivery tracking.

---

4. Project Objectives

The objectives of the project were to:

- build a full-stack food delivery platform
- implement secure authentication and role-based authorization
- allow customers to browse restaurants and place orders
- allow restaurant users to manage their own restaurant operations
- allow drivers to accept and complete delivery tasks
- maintain data persistence using PostgreSQL
- simulate a realistic delivery workflow through a multi-app architecture
- create a modular and presentation-ready software solution

---

5. Scope of the System

In Scope

- user authentication
- role-based access control
- seeded demo users and restaurants
- customer restaurant browsing
- customer order placement
- restaurant-specific order viewing
- restaurant dashboard and menu/availability support
- driver delivery workflow
- order and delivery status updates
- PostgreSQL data persistence
- local development setup and execution

Out of Scope

- live GPS map tracking
- online payments
- push notifications
- SMS/email notifications
- advanced admin management
- production cloud deployment

- multi-tenant enterprise scaling

---

6. Functional Requirements

6.1 Customer Functional Requirements

The customer must be able to:

- log in to the customer application
- browse available restaurants
- view menu items for a selected restaurant
- add items to cart
- place an order
- provide/select a delivery address
- view order status updates
- track delivery progress

6.2 Restaurant Functional Requirements

The restaurant user must be able to:

- log in to the restaurant console
- access only their assigned restaurant
- view incoming restaurant orders
- monitor order progress
- manage restaurant availability-related features
- manage restaurant menu-related features
- view restaurant summary cards and operational data

6.3 Driver Functional Requirements

The driver must be able to:

- log in to the driver application
- view available deliveries
- accept a delivery
- view current deliveries
- update delivery state to picked up
- update delivery state to en route
- update delivery state to delivered
- view completed deliveries

6.4 Backend Functional Requirements

The backend must:

- authenticate users securely
- issue valid JWT tokens
- enforce authorization per role
- persist users, restaurants, menus, orders, and deliveries
- expose REST endpoints for each application
- support seeding of demo data
- maintain restaurant ownership isolation
- link delivery flow to the order flow

---

7. Non-Functional Requirements

The system should:

- be modular and maintainable
- provide acceptable response times for local usage
- separate concerns by role and application
- enforce secure protected routes and API access
- store persistent data reliably in PostgreSQL
- provide clear and usable interfaces
- be testable in a local development environment
- be extensible for future features such as notifications, payments, and analytics

---

8. Technology Stack

Backend

- Java
- Spring Boot
- Spring Security
- Spring Data JPA
- Hibernate
- PostgreSQL
- Maven

Frontend

- Next.js

- React
- TypeScript
- Tailwind CSS
- Axios
- Zustand
- js-cookie

Development Tools

- VS Code
- PowerShell / terminal
- npm
- Maven CLI

---

9. System Architecture

The system follows a **client-server architecture** with separated frontend clients for each operational role.

Architecture Components

1. Backend API
The backend serves as the central processing layer. It handles: - authentication - authorization - business logic - data persistence - API responses - entity relationships - delivery and order coordination

2. Customer Web App
Used by customers to browse restaurants, choose menu items, place orders, and monitor delivery progress.

3. Restaurant Console
Used by restaurant staff or owners to log in and manage restaurant-specific activities such as incoming orders and restaurant operations.

4. Driver App
Used by drivers to see available deliveries, accept jobs, and update delivery statuses.

5. Database
PostgreSQL stores all persistent entities such as users, restaurants, addresses, menu categories, menu items, orders, order items, deliveries, and driver profiles.

Architectural Rationale

A single backend with multiple role-specific frontend applications was chosen because it:

- keeps business logic centralized
- creates cleaner user experiences per role
- makes security and authorization easier to enforce
- reduces frontend complexity by tailoring each app to one responsibility
- better reflects how real delivery platforms operate

---

10. High-Level Architecture Description

A simplified text description of the architecture is as follows:

- the **customer app** sends requests to the backend for restaurants, menus, cart checkout, and order placement
- the **restaurant console** sends requests to the backend for restaurant-specific data, operational controls, and restaurant orders
- the **driver app** sends requests to the backend for available deliveries, delivery acceptance, and status updates
- the **backend** validates tokens, applies business rules, reads/writes from PostgreSQL, and returns data to each frontend
- the **database** stores persistent information used by all applications

---

11. Project Structure

A clean high-level structure of the project is:

```
food-delivery-platform/
├── backend/
├── frontend/
│   ├── customer-web/
│   ├── driver-app/
│   └── restaurant-console/
└── README.md
```

Example: Restaurant Console Structure

```
frontend/
└── restaurant-console/
    ├── src/
    │   ├── app/
    │   │   ├── login/
    │   │   │   └── page.tsx
    │   │   ├── globals.css
    │   │   ├── layout.tsx
    │   │   └── page.tsx
    │   ├── components/
    │   │   ├── AvailabilityToggle.tsx
```

```
│   │   ├── OrderCard.tsx
│   │   ├── OrderDetailModal.tsx
│   │   ├── PrepTimeSelector.tsx
│   │   ├── StatsCard.tsx
│   │   └── StatusBadge.tsx
│   ├── lib/
│   │   ├── api.ts
│   │   ├── mockData.ts
│   │   └── utils.ts
│   ├── store/
│   │   ├── menuStore.ts
│   │   ├── ordersStore.ts
│   │   └── restaurantStore.ts
│   └── types/
│       └── index.ts
├── next.config.js
├── package.json
├── postcss.config.js
├── tailwind.config.ts
└── tsconfig.json
```

---

12. Authentication and Authorization

Authentication

The system uses JWT-based authentication.

When a user logs in: - the frontend sends email and password to the backend - the backend validates the credentials - the backend returns a JWT token - the frontend stores the token and attaches it to protected requests

Roles

The core roles are: - `CUSTOMER` - `RESTAURANT_STAFF` - `DRIVER`

Authorization

Protected backend endpoints are restricted according to role. This prevents: - customers from accessing restaurant endpoints - restaurant users from accessing driver operations - drivers from accessing customer-only or restaurant-only operations

Ownership Restriction

A key design requirement was ensuring restaurant users only see their own restaurant data. The final seeded setup assigns: - Burger Palace to burger.owner@test.com - Pizza Corner to pizza.owner@test.com - Sushi Spot to sushi.owner@test.com

This supports proper data isolation between restaurant accounts.

---

13. Database Design Overview

The database supports all major platform operations. Core data includes:

- users
- addresses
- restaurants
- menu categories
- menu items
- orders
- order items
- deliveries
- driver profiles

The design is relational and structured to maintain referential integrity between users, restaurants, orders, and deliveries.

---

14. Entity Relationship Design (ERD) Explanation

Main Entities

14.1 User

Represents a system user. A user may be a customer, restaurant staff, or driver.

Important fields include: - id - email - password - firstName - lastName - phone - role

14.2 Address

Represents a customer delivery address.

Relationship: - one user can have one or more addresses

14.3 Restaurant

Represents a restaurant on the platform.

Relationship: - a restaurant belongs to one restaurant owner/staff user - one owner can be associated with a restaurant

Important fields include: - name - description - cuisineType - location details - owner - active/open flags

14.4 MenuCategory

Represents a menu section for a restaurant, such as Burgers, Drinks, Pizza, or Sushi.

Relationship: - one restaurant has many menu categories

14.5 MenuItem

Represents a food or beverage item in a menu category.

Relationship: - one menu category has many menu items

14.6 Order

Represents a customer order.

Relationship: - one customer can place many orders - one restaurant can receive many orders

14.7 OrderItem

Represents an individual line item inside an order.

Relationship: - one order has many order items - each order item references a menu item

14.8 Delivery

Represents the delivery task associated with an order.

Relationship: - one order can be linked to one delivery flow - one driver can perform many deliveries over time

14.9 DriverProfile

Represents driver-specific operational data.

Relationship: - one driver profile belongs to one driver user

Relationship Summary

- User → Address = one-to-many
- User → Restaurant = one-to-many or one-to-one depending on implementation approach
- Restaurant → MenuCategory = one-to-many
- MenuCategory → MenuItem = one-to-many

- User (Customer) → Order = one-to-many
- Restaurant → Order = one-to-many
- Order → OrderItem = one-to-many
- Order → Delivery = one-to-one or one-to-many depending on model design, but conceptually one delivery lifecycle per order
- User (Driver) → DriverProfile = one-to-one
- User (Driver) → Delivery = one-to-many over time

Strong and Weak Relationship Discussion

A **strong relationship** exists where the child entity depends heavily on the parent entity for meaning. Examples include: - Order → OrderItem - Restaurant → MenuCategory - MenuCategory → MenuItem

A **weaker or more loosely coupled relationship** exists where an entity may conceptually stand independently but is linked operationally. Examples include: - User → Address - User → DriverProfile - Order → Delivery

In the delivery platform, the strongest lifecycle dependency is the order structure itself. Order items have no practical meaning without the order they belong to.

---

15. Backend Design Explanation

The backend is built using Spring Boot and organized around domain-driven modules.

Main Backend Responsibilities

- authenticating users
- issuing JWTs
- managing restaurants and menu data
- processing orders
- coordinating deliveries
- storing and retrieving data using JPA repositories
- applying role-based security

Common Backend Layers

Controllers

Controllers expose REST endpoints for frontend applications.

Services

Services contain business logic such as: - processing orders - assigning deliveries - filtering restaurant-specific data - updating delivery states

Repositories

Repositories provide database access using Spring Data JPA.

Entities

Entities define the database model and relationships.

Configuration

Configuration classes manage: - security - CORS - startup seed data

Key Design Benefits

- separation of concerns
- clear domain boundaries
- easier debugging and testing
- cleaner integration with multiple frontends

---

16. Frontend Design Explanation

The frontend is intentionally split into role-specific applications.

16.1 Customer Web

Designed for customer-focused tasks: - browsing restaurants - viewing menus - ordering food - tracking order progress

16.2 Restaurant Console

Designed for restaurant operations: - login for restaurant staff - restaurant-only dashboard - order monitoring - menu and availability support - restaurant metrics cards

16.3 Driver App

Designed for delivery operations: - login for drivers - available deliveries list - current deliveries list - completed deliveries list - action buttons for status progression

Frontend State and API Design

The frontends use: - Axios for HTTP requests - Zustand for state management in selected modules - cookies for token storage where applicable - Next.js app routing for page structure

This provides a modern and modular frontend setup while keeping role experiences independent.

---

17. Data Seeder and Demo Data

A `DataSeeder` was used to make the application testable immediately after backend startup.

The seed data includes: - a test customer - a test driver - a driver profile - three different restaurant owner accounts - three restaurants - menu categories - menu items - a test delivery address

Final Demo Accounts

Customer

- `test@test.com / 12345678`

Driver

- `driver@test.com / 12345678`

Restaurant Owners

- `burger.owner@test.com / 12345678`
- `pizza.owner@test.com / 12345678`
- `sushi.owner@test.com / 12345678`

Demo Data Benefit

Seeding made it possible to quickly verify the system without manually creating test users and restaurants each time.

---

18. Order Lifecycle

The order lifecycle in the system can be summarized as follows:

1. Customer logs in
2. Customer browses restaurants
3. Customer selects menu items
4. Customer places an order

5. Backend stores the order and order items
6. Restaurant owner sees the order for the relevant restaurant
7. Delivery becomes available to a driver
8. Driver accepts the delivery
9. Driver updates the delivery through operational states
10. Order reaches completion once delivery is marked delivered

This lifecycle demonstrates the coordination between customer, restaurant, driver, and backend modules.

---

19. Delivery Lifecycle

The delivery lifecycle is central to the driver application.

A typical delivery flow is:

1. Delivery created or exposed as available
2. Driver accepts delivery
3. Driver status becomes assigned
4. Driver marks picked up
5. Driver marks en route
6. Driver marks delivered

These transitions are reflected in the UI and supported by backend endpoints.

---

20. Sequence Explanation

A simplified interaction sequence is:

Customer Places Order

- customer frontend sends order request
- backend validates token and request payload
- backend creates order and order items
- backend stores order in database
- backend returns order confirmation

Restaurant Views Order

- restaurant console sends authenticated request
- backend identifies logged-in restaurant owner
- backend fetches only that restaurant’s orders

- backend returns relevant order data

Driver Accepts Delivery

- driver app requests available deliveries
- backend returns unassigned deliveries
- driver accepts one delivery
- backend assigns the delivery to the driver
- delivery moves into current deliveries

Driver Completes Delivery

- driver app sends pickup/en-route/delivered updates
- backend updates delivery status in the database
- frontend views reflect the latest status

---

21. Security Design

The system incorporates the following security-oriented measures:

- Spring Security configuration
- JWT token authentication
- route protection for authenticated users
- role-based authorization for API endpoints
- CORS configuration for frontend apps on local ports
- token-based access for protected frontend requests

CORS Consideration

Because different frontend apps run on different localhost ports, the backend CORS configuration was updated to allow those origins, including the driver app port.

This was necessary for successful communication between the backend and all frontend applications.

---

22. Testing Strategy

Approach

Testing was performed primarily as functional and integration testing through realistic user flows.

Major Test Scenarios

Authentication Tests

- customer login works
- restaurant owner login works
- driver login works
- invalid or unauthorized access is restricted

Restaurant Isolation Tests

- burger owner sees Burger Palace only
- pizza owner sees Pizza Corner only
- sushi owner sees Sushi Spot only
- one owner cannot access another owner's restaurant data through normal flow

Customer Tests

- customer can browse restaurants
- customer can see menus
- customer can add items to cart
- customer can place an order successfully

Driver Tests

- driver can view available deliveries
- driver can accept a delivery
- driver can update delivery states
- driver can view completed deliveries

End-to-End Integration Tests

- customer places order at a restaurant
- the correct restaurant sees the correct order
- the driver sees the delivery
- the driver accepts and completes the delivery
- status changes propagate successfully through the system

Testing Result

The end-to-end system test was successful.

That confirms that the core workflow across customer, restaurant, driver, backend, and database is functioning correctly.

---

23. Final Verification Summary

The following has been verified:

- backend startup and database seeding
- separate seeded owners for separate restaurants
- working customer login
- working restaurant login
- working driver login
- successful customer order placement
- restaurant-specific visibility of orders
- driver delivery acceptance and progression
- successful cross-app status flow

This means the platform is functionally complete for demonstration and academic/project submission purposes.

---

24. Challenges Encountered and Resolutions

Challenge 1: Hardcoded Restaurant Behavior

Initially, the system behavior was tied too closely to specific hardcoded restaurant assumptions.

Resolution:
Restaurant ownership and lookup logic were adjusted so that logged-in restaurant users could be associated with different restaurants instead of a single hardcoded one.

Challenge 2: Seeder Ownership and Duplicate Data Issues

When multiple owners were introduced, existing phone number uniqueness constraints caused startup failures.

Resolution:
Unique phone numbers were assigned to the new seeded owner accounts, and restaurant ownership reassignment logic was updated.

Challenge 3: Driver App Network Error

The driver app could not log in even though the backend was running.

Resolution:
The backend CORS configuration was updated to include the actual frontend origin used by the driver app.

Challenge 4: Role-Based Testing Across Multiple Frontends

Managing multiple apps and terminals can easily create confusion during testing.

Resolution:
A structured test flow was used, with separate terminals and clearly seeded accounts for each role.

---

25. UI and Final Polish Summary

The final system includes role-specific UI components such as:

Restaurant Console UI Elements

- status badges
- summary metrics cards
- order cards
- order detail modal
- preparation time selector
- availability controls

Driver App UI Elements

- available delivery panels
- current delivery panels
- completed delivery panels
- action buttons for state changes
- delivery performance summary cards

Customer UI Elements

- restaurant browsing views
- cart and checkout flow
- order placement and tracking views

The user experience is now significantly more coherent, modular, and aligned with the expected behavior of a modern delivery platform.

---

26. How to Run the Project

Prerequisites

Install the following: - Java 17 or later - Maven - Node.js and npm - PostgreSQL

Backend

From the project root:

```
cd backend
mvn spring-boot:run
```

Expected backend URL:

```
http://localhost:8080
```

Restaurant Console

```
cd frontend/restaurant-console
npm install
npm run dev
```

Customer Web

```
cd frontend/customer-web
npm install
npm run dev
```

Driver App

```
cd frontend/driver-app
npm install
npm run dev
```

The frontend applications will run on available localhost ports, such as 3000, 3001, 3002, or 3003.

---

27. Demo Login Credentials

Customer

- `test@test.com`
- `12345678`

Driver

- `driver@test.com`
- `12345678`

Restaurant Owners

- `burger.owner@test.com`
- `12345678`
- `pizza.owner@test.com`
- `12345678`
- `sushi.owner@test.com`

- 12345678

---

28. README Version

The following README text can be used directly in the project root.

```
# Food Delivery Platform

A full-stack food delivery platform inspired by services like Mr D, built
with a Spring Boot backend and separate frontend applications for customers,
restaurant owners, and drivers.

## Overview

This project simulates a real-world food delivery ecosystem where:

- customers browse restaurants and place orders
- restaurant owners manage restaurant-specific orders and menu operations
- drivers accept and complete delivery jobs
- the backend coordinates authentication, order management, restaurant
operations, and delivery workflows

The system was designed as a role-based platform with separate interfaces for
each operational user group.

## Key Features

### Customer
- Browse available restaurants
- View menus and menu items
- Add items to cart
- Place food orders
- Track order and delivery progress

### Restaurant Console
- Restaurant staff login
- View restaurant-specific orders
- Manage menus and availability
- Monitor restaurant order flow
- View dashboard summary cards

### Driver App
- Driver login
- View available delivery jobs
- Accept deliveries
- Update delivery status
- View current and completed deliveries
```

Backend
- JWT-based authentication
- Role-based access control
- PostgreSQL persistence
- Seeded demo users, restaurants, menu categories, and menu items
- REST API for customer, restaurant, and driver operations

Tech Stack

Backend
- Java
- Spring Boot
- Spring Security
- Spring Data JPA
- Hibernate
- PostgreSQL
- Maven

Frontend
- Next.js
- React
- TypeScript
- Tailwind CSS
- Zustand
- Axios
- js-cookie

System Modules

The platform is divided into four main parts:

1. **Backend**
   - handles business logic, authentication, database access, and APIs

2. **Customer Web**
   - allows customers to browse restaurants and place orders

3. **Restaurant Console**
   - allows restaurant owners/staff to manage restaurant operations

4. **Driver App**
   - allows drivers to manage delivery assignments and delivery progress

Project Structure

```text
food-delivery-platform/
```

```
├── backend/
├── frontend/
│   ├── customer-web/
│   ├── driver-app/
│   └── restaurant-console/
└── README.md
```

Authentication and Roles

The application uses role-based authentication.

Roles

- `CUSTOMER`
- `RESTAURANT_STAFF`
- `DRIVER`

Authentication

- login is handled by the backend through JWT authentication
- frontend apps store role-specific tokens for authenticated requests
- protected endpoints enforce access based on user role

Demo Accounts

Customer

- `test@test.com / 12345678`

Driver

- `driver@test.com / 12345678`

Restaurant Owners

- `burger.owner@test.com / 12345678`
- `pizza.owner@test.com / 12345678`
- `sushi.owner@test.com / 12345678`

Setup Requirements

Before running the project, make sure you have:

- Java 17+
- Maven
- Node.js and npm
- PostgreSQL
- VS Code or another IDE/editor

Backend Setup

1. Open a terminal in the backend folder:

   ```
   cd backend
   ```

2. Run the Spring Boot application:

   ```
   mvn spring-boot:run
   ```

3. The backend should start on:

   ```
   http://localhost:8080
   ```

Frontend Setup

Restaurant Console

```
cd frontend/restaurant-console
npm install
npm run dev
```

Customer Web

```
cd frontend/customer-web
npm install
npm run dev
```

Driver App

```
cd frontend/driver-app
npm install
npm run dev
```

Core Workflow

Customer Flow

1. Customer logs in
2. Customer browses restaurants
3. Customer selects menu items
4. Customer places an order
5. Customer tracks the order and delivery status

Restaurant Flow

1. Restaurant owner logs in
2. Owner sees only their restaurant
3. Owner monitors incoming orders
4. Restaurant prepares the order
5. Delivery is handed over for driver assignment

Driver Flow

1. Driver logs in
2. Driver views available deliveries
3. Driver accepts a delivery job
4. Driver marks delivery as picked up
5. Driver marks delivery as en route
6. Driver marks delivery as delivered

Functional Testing Summary

The platform was tested successfully through an end-to-end workflow:

- customer login works
- restaurant owner login works
- driver login works
- customer can place an order
- correct restaurant owner sees the correct order
- driver sees and accepts available delivery jobs
- driver updates delivery statuses successfully
- customer sees updated delivery progress
- restaurant owners are separated by assigned restaurant ownership

Future Improvements

- admin dashboard
- payment gateway integration
- real-time notifications
- live order tracking with websockets
- driver location tracking on maps
- restaurant analytics dashboard
- customer order history improvements
- search, filter, and sorting enhancements
- stronger validation and error handling
- deployment to cloud hosting

License

This project is for demonstration purposes.